DYP DPU

Dr. D. Y. Patil Institute of Technology, Pimpri, Pune

Department of Information Technology

LABORATORY MANUAL

LAB-PRACTICE-V (Distributed System)

SEMESTER-VIII

Subject Code: **414454**

TEACHING SCHEME
Lectures: 4Hrs/Week
Practical: 4 Hrs/Week

EXAMINATION SCHEME
Practical: 25 Marks
Term Work: 50 Marks

Name of Faculty
Prof. Maheshwari S. Divate

Final Year Information Technology (2019 Course)
414454: Lab Practice – V

Teaching Scheme:	Credit Scheme:	Examination Scheme:
Practical (PR): 4 hrs/week	02 Credits	PR: 25 Marks TW: 50 Marks

Prerequisites:
1. **Operating Systems**
2. **Computer Network Technology**
3. Web Application Development

Course Objectives:
1. The course aims to provide an understanding of the principles on which the distributed systems are based, their architecture, algorithms and how they meet the demands of Distributed applications.
2. The course covers the building blocks for a study related to the design and the implementation of distributed systems and applications.

Course Outcomes:
Upon successful completion of this course student will be able to:
1. Demonstrate knowledge of the core concepts and techniques in distributed systems.
2. Learn how to apply principles of state-of-the-Art Distributed systems in practical application.
3. **Design, build and test application programs on distributed systems**

Guidelines for Student's Lab Journal
1. The laboratory assignments are to be submitted by students in the form of journals. The Journal consists of prologue, Certificate, table of contents, and handwritten/printed write-up of each assignment (Title, Objectives, Problem Statement, Outcomes, Software & Hardware requirements, Date of Completion, Assessment grade/marks and assessor's sign, Theory-Concept, algorithms, printouts of the code written using coding standards, sample test cases etc.)
2. **Candidate is expected to know the theory involved in the experiment.**
3. The practical examination should be conducted if the journal of the Student is completed in all respects and certified by concerned faculty and head of the department.

Guidelines for Lab /TW Assessment
The term work based on performance of students considering the parameters such as
1)timely conduction of practical assignment,
2)methodology adopted for implementation of practical assignment,
3)timely submission of assignment in the form of handwritten/printed write-up along with results of implemented assignment,
5)attendance etc.

6) The understanding of the practical performed in the examination by asking some questions relatedto theory & implementation of experiments he/she has carried out.

Guidelines for Laboratory Conduction

All the assignments should be conducted on the latest version of Open-Source Operating Systems, tools and Multi-core CPU supporting Virtualization and Multi-Threading.

List of Laboratory Assignments

1. Implement multi-threaded client/server Process communication using RMI.

2. Develop any distributed application using CORBA to demonstrate object brokering.(Calculator or String operations).

3. Develop a distributed system, to find sum of N elements in an array by distributing N/n elements to n number of processors MPI or OpenMP. Demonstrate by displaying the intermediate sums calculated at different processors.

4. Implement Berkeley algorithm for clock synchronization.

5. Implement token ring based mutual exclusion algorithm.

6. Implement Bully and Ring algorithm for leader election.

7. Create a simple web service and write any distributed application to consume the web service.

8. Mini Project (In group): A Distributed Application for Interactive Multiplayer Games

Reference Books:

1. Distributed Systems –Concept and Design, George Coulouris, Jean Dollimore, Tim Kindberg& GordonBlair,Pearson,5th Edition,ISBN:978-13-214301-1.

2. Distributed Algorithms,Nancy Ann Lynch, Morgan Kaufmann Publishers, illustrated, reprint, ISBN:9781558603486.

3. Java Network Programming & Distributed Computing by David Reilly, Michael Reilly

4. Distributed Systems - An Algorithmic approach by Sukumar Ghosh (good book for distributed algorithms)

5. Distributed Algorithms: Principles, Algorithms, and Systems by A. D. Kshemkalyani and M. Singhal(Good for algorithms, but very detailed, has lots of algorithms; good reference)

6. Design and Analysis of Distributed Algorithms by Nicola Santoro (good, distributed algorithms book)

ASSIGNMENTNO:1

Title:Socket Programming

Aim: To develop anydistributed application through implementing client-server communication programs based on Java Sockets.

Objectives:

1. Tostudyinterprocess communication
2. Tolearnclientserver communicationusingsocket

Tools / Environment:

JavaProgrammingEnvironment,rmiregistry,jdk 1.8,Eclipse IDE.

RelatedTheory:

Socket: In distributed computing, network communication is one of the essential parts of any system, and the socket is the endpoint of every instance of network communication. In Java communication, it is the most critical and basic object involved.

A socket is a handle that a local program can pass to the networking API to connect to another machine. It can be defined as the terminal of a communication link through which two programs /processes/threadsrunningon the networkcancommunicatewith each other.TheTCP layer can easilyidentifythe application location and access information through the port number assigned to the respective sockets.

During an instance of communication, a client program creates a socket at its end and tries to connect it to the socket on the server. When the connection is made, the server creates a socket at its end and then server and client communication is established.

Designing thesolution:

The **java.net** package provides classes to facilitate the functionalities required for networking. The **socket** class programmed through Java using this package has the capacity of being independent of the platform of execution; also, it abstracts the calls specific to the operating system on which it is invoked from other Java interfaces. The **ServerSocket** class offers to observe connection invocations, and it accepts such invocations from different clients through another socket. High-level wrapper classes, such as **URLConnection** and**URLEncoder**, are more appropriate. If you want to establish a connection to the Web using a URL, then these classes will use the socket internally.

The**java.net**packageprovidessupportforthetwocommonnetworkprotocols−

- **TCP** − TCP stands for Transmission Control Protocol, which allows for reliable communication between two applications. TCP is typically used over the Internet Protocol, which is referred to as TCP/IP.

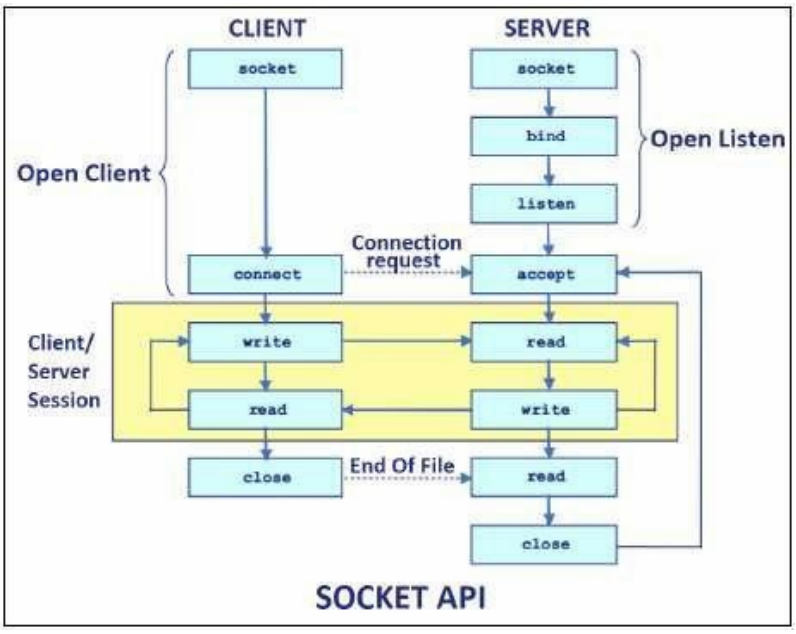
- **UDP** − UDP stands for User Datagram Protocol, a connection-less protocol that allows forpackets of data to be transmitted between applications.

SocketprogrammingforTCP:

The following steps occur when establishing a TCP connection between two computers usingsockets −

- The server instantiates a ServerSocket object, denoting which port number communication is to occur on.
- The server invokes the accept() method of the ServerSocket class. This method waits until a client connects to the server on the given port.
- After the server is waiting, a client instantiates a Socket object, specifying the server name and the port number to connect to.
- The constructor of the Socket class attempts to connect the client to the specified server andtheportnumber. Ifcommunicationisestablished,theclientnowhasa Socketobject capable of communicating with the server.
- On the server side, the accept() method returns a reference to a new socket on the server that is connected to the client's socket.
- After the connections are established, communication can occur using I/O streams. Each socket has both an OutputStream and an InputStream. The client's OutputStream is connected to the server's InputStream, and the client's InputStream is connected to the server's OutputStream.

- TCPisatwo-waycommunicationprotocol,hencedatacanbesentacrossbothstreamsat the same time. Following are the useful classes providing complete set of methods to implement sockets.



SocketprogrammingforUDP:

UDP is used only when the entire information can be bundled into a single packet and there is no dependency on the other packet. Therefore, the usage of UDP is quite limited,whereasTCPiswidelyusedinIPapplications.UDPsocketsareusedwherelimitedbandwidthis available, and the overhead associated with resending packets is not acceptable.

To connect using a UDP socket on a specific port, use the following code:

DatagramSocketudpSock=newDatagramSocket(3000);

A datagram is a self-contained, independent message whose time of arrival, confirmation of arrival over the network, and content cannot be guaranteed. atagramPacket objects are used to senddataoverDatagramSocket.EveryDatagramPacketobjectconsistsofadatabuffer,aremote host to whom the data needs to be sent, and a port number on which the remote agent would be listened.

Implementingthe solution:

SocketProgrammingforTCP:

Client Programming:

1. **Establish a Socket Connection:** The **java.net Socket** classrepresentsaSocket.Toopen a socket:

   Socketsocket=newSocket("127.0.0.1",5000)

2. **Communication:** To communicate over a socket connection, streams are used to both input and output the data.

3. **Closing the connection:** The socket connection is closed explicitly once the message to server is sent.

Server Programming:

1. **Establish a Socket Connection:** Towriteaserverapplicationtwosocketsareneeded.A ServerSocketwhichwaitsfortheclientrequests(whenaclientmakesanewSocket()).A plain old Socket socket to use for communication with the client.

2. **Communication:**getOutputStream()methodisusedtosendtheoutputthrough the socket.

3. **Close the Connection:** After finishing,itisimportanttoclose the connectionbyclosing the socket as well as input/output streams.

CompilationandExecutingthe solution:

Ifyou'reusingEclipse:

1. Compileboth ofthem ontwo differentterminals or tabs

2. RuntheServerprogramfirst

3. ThenruntheClient program

4. TypemessagesintheClientWindowwhichwillbereceivedandshowedbytheServer Window simultaneously if you are developing echo server application.

5. Closethe socket connectionbytyping somethinglike"Exit".

Conclusion:

In this assignment, the students learned about client-server communication through different protocols and sockets. They also learned about Java support through the socket API for TCP and UDP programming.

Outcome:

1. StudentslearnedaboutinterprocesscommunicationinDS
2. Students develop application through implementing client-server communication programs based on Java Sockets

FAQ:

1. Whatisinterprocesscommunication?
2. What is socket?

3. DifferencebetweenTCP andUDPsocket communication?
4. What is shared memoryprogramming?
5. Whatisport? Stateapplicationofport.

ASSIGNMENTNO.2

Title:RemoteMethodInvocation

Aim: Toimplementmulti-threaded client/serverProcesscommunicationusingRMI.

Objectives:

1. Tounderstandthebasicsofmulti-threadedclient/serverProcesscommunication.
2. To develop interprocess communication application using RMI.

Tools / Environment:

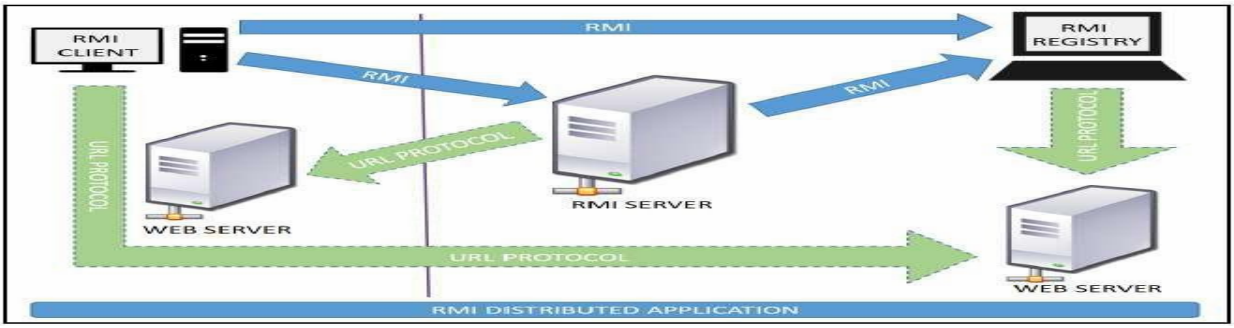
JavaProgrammingEnvironment,jdk 1.8, rmiregistry

RelatedTheory:

RMI provides communication between java applications that are deployed on different servers andconnectedremotelyusingobjectscalled **stub**and**skeleton**.Thiscommunicationarchitecture makes a distributed application seem like a group of objects communicating across a remote connection. These objects are encapsulated by exposing an interface, which helps access the private state and behavior of an object through its methods.

The following diagram shows how RMI happens between the RMI client and RMI server with the help of the

RMIregistry:



RMI REGISTRY is a remote object registry, a Bootstrap naming service, that is used by **RMI SERVER** on the same host to bind remote objects to names. Clients on local and remote hosts then look up the remote objects and make remote method invocations.

KeyterminologiesofRMI:

The following are some of the important terminologies used in a Remote Method Invocation. **Remote object**: This is an object in aspecificJVMwhosemethods areexposedso theycould be invoked by another program deployed on a different JVM.

Remote interface: This is a Java interface that defines the methods that exist in a remote object. A remote object can implement more than one remote interface to adopt multiple remote interface behaviors.

RMI: This is a wayof invoking a remote object's methods with the help of a remote interface. It can be carried with a syntax that is similar to the local method invocation.

Stub: This is a Java object that acts as an entry point for the client object to route any outgoing requests. It exists on the client JVM and represents the handle to the remote object.

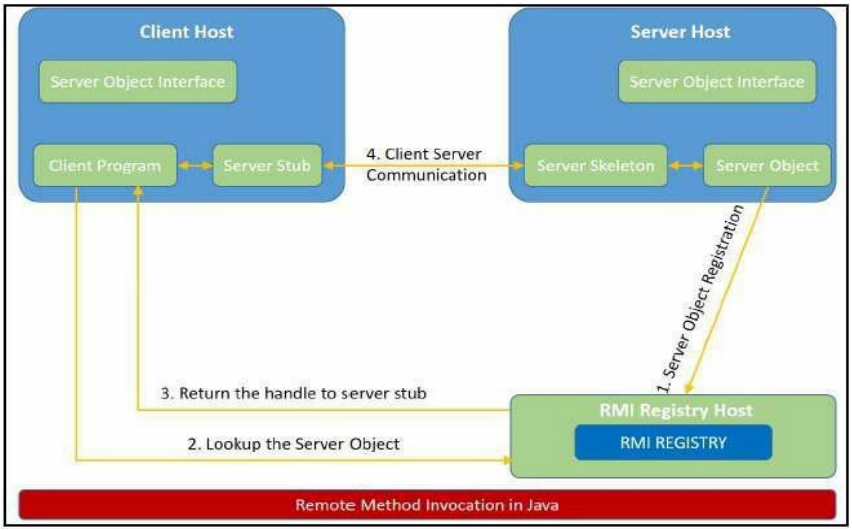
If any object invokes a method on the stub object, the stub establishes RMI by following these steps:

1. ItinitiatesaconnectiontotheremotemachineJVM.
2. Itmarshals (write andtransmit)theparameterspassedtoitviatheremoteJVM.
3. Itwaitsfor aresponsefromtheremoteobjectand unmarshals(read) thereturnedvalue or exception, then it responds to the caller with that value or exception.

Skeleton: This is an object that behaves like a gateway on the server side. It acts as a remote object with which the client objects interact through the stub. This means that any requests coming from the remote client are routed through it. If the skeleton receives a request, it establishes RMI through these steps:

1. Itreadstheparametersenttotheremotemethod.
2. Itinvokesthe actualremoteobjectmethod.
3. Itmarshals(writes andtransmits)theresultbacktothecaller (stub).

Thefollowingdiagram demonstrates RMIcommunicationwith stub and skeleton involved:



Designing theSolution:

Theessentialstepsthatneedtobefollowedtodevelopadistributedapplicationwith RMIareas follows:

1. Designandimplementacomponentthatshouldnotonlybeinvolvedin thedistributed application, but also the local components.

2. Ensurethatthecomponentsthat participateinthe RMIcallsare accessible across networks.

3. Establishanetworkconnection betweenapplicationsthatneedto interactusingthe RMI.

Remote interface definition: The purpose of defining a remote interface is to declare the methods that should be available for invocation by a remote client.

Programming the interface instead of programming the component implementation is anessential design principle adopted by all modern Java frameworks, including Spring. In the same pattern, the definition of a remote interface takes importance in RMI design as well.

2. **Remote object implementation**: Java allows a class to implement more than one interface ata time. This helps remote objects implement one or more remote interfaces. The remote object class may have to implement other local interfaces and methods that it is responsible for. Avoid adding complexity to this scenario, in terms of how the arguments or return parameter values of such component methods should be written.

3. **Remote client implementation**: Client objects that interact with remote server objects can be written once the remote interfaces are carefully defined even after the remote objects are deployed.

Let's design a project that can sit on a server. After that different client projects interact with this project to pass the parameters and get the computation on the remote object execute and return the result to the client components.

Implementingthe solution:

Considerbuildinganapplicationtoperformdiversemathematical operations.

Theserverreceivesarequestfromaclient,processesit,andreturnsaresult.Inthisexample,the request specifies two numbers. The server adds these together and returns the sum.

1. Creatingremoteinterface,implementremoteinterface,server-sideandclient-side program and Compile the code.

This application uses four source files. The first file, **AddServerIntf.java**, defines the remote interface that is provided by the server. It contains one method that accepts two **double** arguments and returns their sum. All remote interfaces must extend the **Remote** interface, which is part of **java.rmi**. **Remote** defines no members. Its purpose is simply to indicate that an interface uses remote methods. All remote methods can throw a **RemoteException**.

The second source file, **AddServerImpl.java**, implements the remote interface. The implementation of the **add()** method is straightforward. All remote objects must extend **UnicastRemoteObject**, which provides functionality that is needed to make objects available from remote machines.

The third source file, **AddServer.java**, contains the main program for the server machine. Its primary function is **to update the RMI registry on that machine**. This is done by using the **rebind()** method of the **Naming** class (found in **java.rmi**). That method associates a name with an object reference. The first argument to the **rebind()** method is a string that names the

server as "AddServer". Its second argument is a reference to an instance of

AddServerImpl.Thefourthsource file,**AddClient.java**,implements the clientside of

thisdistributedapplication.
AddClient.javarequiresthreecommand-linearguments.ThefirstistheIPaddressornameof theservermachine.Thesecondandthird argumentsarethe twonumbersthat aretobe summed.

The application begins by forming a string that follows the URL syntax. This URL uses the **rmi** protocol. The string includes the IP address or name of the server and the string "AddServer". The program then invokes the **lookup( )** method of the **Naming** class. This method accepts one

argument, the **rmi** URL, and returns a reference to an object of type **AddServerIntf**. All remote method invocations can then be directed to this object.

Theprogramcontinuesbydisplayingits argumentsandtheninvokes theremote**add()**method.

The sum is returned from this method and is then printed.

Use**javac**tocompilethefoursourcefilesthat arecreated.

2. GenerateaStub

Before using client and server, the necessary stub must be generated. In the context of RMI, a *stub* is a Java object that resides on the client machine. Its function is to present the same interfaces as the remote server. Remote method calls initiated by the client are actually directed to the stub. The stub works with the other parts of the RMI system to formulate a request that is sent to the remote machine.

All of this information must be sent to the remote machine. That is, an object passed as an argument to a remote method call must be serialized and sent to the remote machine. If a response must be returned to the client, the process works in reverse. **The serialization and deserialization facilities are also used if objects are returned to a client.**

Togenerateastubthecommand isRMIcompilerisinvokedas follows:

rmicAddServerImpl.

Thiscommandgenerates thefile**AddServerImpl_Stub.class**.

3. InstallFilesontheClientandServer Machines

Copy **AddClient.class**, **AddServerImpl_Stub.class**, **AddServerIntf.class** to a directory on the client machine.

CopyAddServerIntf.class,AddServerImpl.class,AddServerImpl_Stub.class,and **AddServer.class** to adirectoryonthe server machine.

4. StarttheRMIRegistry ontheServerMachine

Java provides a program called **rmiregistry**, which executes on the server machine. It maps names to object references. Start the RMI Registry from the command line, as shown here:

startrmiregistry

5. Startthe Server

Theservercodeisstarted from thecommandline,asshown here:

java AddServer

The **AddServer** code instantiates **AddServerImpl** and registers that object with the name

“AddServer”.

6. Startthe Client

The **AddClient** software requires three arguments: the name or IP address of the server machine and the two numbers that are to be summed together. You may invoke it from the command line by using one of the two formats shown here:

javaAddClient 192.168.13.14 7 8

Conclusion:

Remote Method Invocation (RMI) allows you to build Java applications that are distributed among several machines. Remote Method Invocation (RMI) allows a Java object that executes on one machine to invoke a method of a Java object that executes on another machine. This is an important feature, because it allows you to build distributed applications.

Outcome:

1. Studentslearnbasicsofmulti-threadedclientserverprocesscommunication.
2. StudentsdevelopinterprocesscommunicationapplicationusingJAVARMI.

FAQ:

1. What is Heterogeneity?
2. Exampleof ismarshalingandunmarshalling?
3. Explain RMIwithdiagram.
4. What is binding?
5. What is roleof RMIregistry?whywestart RMIregistryfirst.
6. What is use of"UnicastRemoteObject","lookup(),rebind().
7. Whatis stub andskeleton?
8. WhatisdifferencebetweenExceptionandRemoteException

ASSIGNMENTNO.3

Title:CommonObjectRequestBrokerArchitecture(CORBA)

Aim: TodevelopanydistributedapplicationwithCORBA programusingJAVAIDL.

Objective:

1. Tounderstandthe basicsstepsforimplementationof CORBA.
2. Todevelopanydistributed applicationusingCORBA todemonstrateobject brokering.

Tools / Environment:

JavaProgrammingEnvironment, JDK1.8

RelatedTheory:

CommonObjectRequestBrokerArchitecture (CORBA):

CORBA is an acronym for Common Object Request Broker Architecture. It is an open source, vendor-independent architecture and infrastructure developed by the **Object Management Group** (**OMG**) to integrate enterprise applications across a distributed network. CORBA specifications provideguidelinesforsuchintegration applications,based onthewaytheywant to interact, irrespective of the technology; hence, all kinds of technologies can implement these standards using their own technical implementations.

When two applications/systems in a distributed environment interact with each other, there are quite a few unknowns between those applications/systems, including the technology they are developed in (such as Java/ PHP/ .NET), the base operating system they are running on (such as Windows/Linux), or system configuration (such as memory allocation). **They communicate mostly with the help of each other's network address or through a naming service.** Due to this, these applications end up with quite a few issues in integration, includingcontent (message) mapping mismatches.

An application developed based on CORBA standards with standard **Internet Inter-ORB Protocol** (**IIOP**),irrespectiveofthevendorthat develops it, should beable to smoothlyintegrate and operate with another application developed based on CORBA standards through the same or different vendor.

Except legacy applications, most of the applications follow common standards when it comes to object modeling,forexample.Allapplications relatedto, say,"HR&Benefits"maintainanobject model with details of the organization, employees with demographic information, benefits, payroll, and deductions. They are only different in the way they handle the details, based on the

country and region they are operating for. For each object type, similar to the HR&Benefits systems, we can define an interface using the **Interface Definition Language** (**OMG IDL**).

The contract between these applications is defined in terms of an interface for the server objects that the clients can call. This IDL interface is used by each client to indicate when they should call any particular method to marshal (read and send the arguments).

The target object is going to use the same interface definition when it receives the request from theclienttounmarshal(readthearguments)inordertoexecutethemethodthatwasrequestedby the client operation. Again, during response handling, the interface definition is helpful to marshal (send from the server) and unmarshal (receive and read the response) arguments on the client side once received.

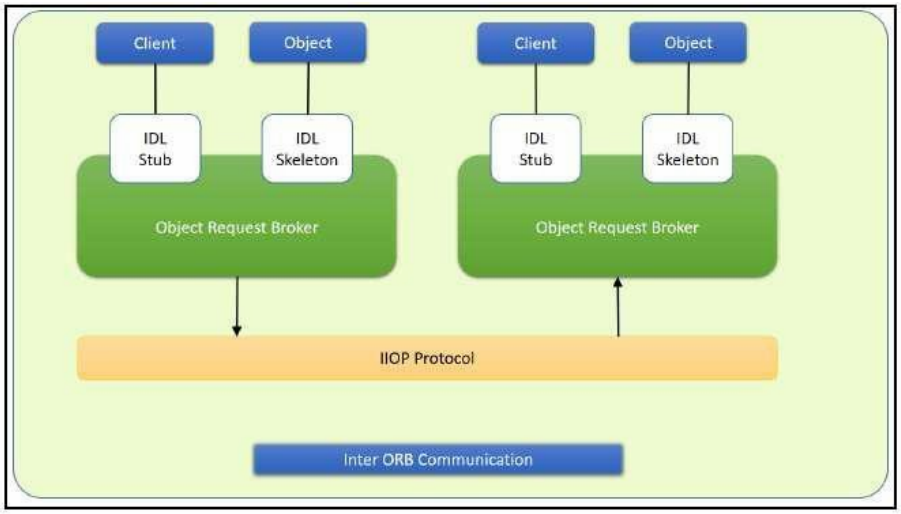
The IDL interface is a design concept that works with multiple programming languagesincluding C, C++, Java, Ruby, Python, and IDLscript. This is close to writing a program to an interface, a concept we have been discussing that most recent programming languages and frameworks, such as Spring. The interface has to be defined clearly for each object. The systems encapsulate the actual implementation along with their respective data handling and processing, and only the methods are available to the rest of the world through the interface. Hence, the clients are forced to develop their invocation logic for the IDL interface exposed by the applicationtheywanttoconnecttowiththemethodparameters(input and output)advisedbythe interface operation.

The following diagram shows a single-process ORB CORBA architecture with the IDL configuredasclientstubswithobjectskeletons,Theobjectsarewritten(ontheright)andaclient for it (on the left), as represented in the diagram. The client and server use stubs and skeletons as proxies, respectively. The IDL interface follows a strict definition, and even though the clientand server are implemented in different technologies, they should integrate smoothly with the interface definition strictly implemented.

In CORBA, each object instance acquires an object reference for itself with the electronic token identifier. Client invocations are going to use these object references that have the ability to figure out which ORB instance they are supposed to interact with. The stub and skeleton represent the client and server, respectively, to their counterparts. They help establish this communication through ORB and pass the arguments to the right method and its instance during the invocation.

Inter-ORBcommunication

The following diagram shows how remote invocation works for inter-ORB communication. It shows that the clients that interacted have created **IDL Stub** and **IDL Skeleton** based on **Object Request Broker** and communicated through **IIOP Protocol**.



To invoke the remote object instance, the client can get its object reference using a naming service. Replacing the object reference with the remote object reference, the client can make the invocation of the remote method with the same syntax as the local object method invocation. ORB keeps the responsibility of recognizing the remote object reference based on the client object invocation through a naming service and routes it accordingly.

JavaSupportforCORBA

CORBAcomplementstheJava™platformbyprovidingadistributedobjectframework,services to support that framework, and interoperability with other languages. The Java platform complements CORBA by providing a portable, highly productive implementation environment, and a very robust platform. By combining the Java platform with CORBA and other key

enterprisetechnologies,theJavaPlatformistheultimateplatformfordistributedtechnology solutions.

CORBA standards provide the proven, interoperable infrastructure to the Java platform. IIOP (Internet Inter-ORB Protocol) manages the communication between the object components that power the system. The Java platform provides a portable object infrastructure that works on every major operating system. CORBA provides the network transparency, Java provides the implementation transparency. **An *Object Request Broker (ORB)* is part of the Java Platform.**

The ORB is a runtime component that can be used for distributed computing using IIOP communication. Java IDL is a Java API for interoperability and integration with CORBA. Java IDL included both a Java-based ORB, which supported IIOP, and the **IDL-to-Java**

compiler,for generatingclient-sidestubsandserver-sidecodeskeletons.J2SEv.1.4includes an

Object Request Broker Daemon (ORBD), which is used to enable clients to

transparently locate and invoke persistent objects on servers in the CORBA environment.

When using the **IDL programming model,** the interface is everything! It defines the points of entry that can be called from a remote process, such as the types of arguments the called procedure will accept, or the value/output parameter of information returned. Using IDL, the programmer can make the entry points and data types that pass between communicating processes act like a standard language.

CORBA is a language-neutral system in which the argument values or return values are limited to what can be represented in the involved implementation languages. In CORBA, object orientationislimitedonlytoobjectsthatcanbepassedbyreference(theobjectcodeitselfcannot be passed from machine-to-machine) or are predefined in the overall framework. Passed and returned types must be those declared in the interface.

WithRMI, the interface and the implementation language aredescribed in the same language, so you don't have to worry about mapping from one to the other. Language-level objects (the code itself) can be passed from one process to the next. Values can be returned by their actual type,not the declared type. Or, you can compile the interfaces to generate IIOP stubs and skeletons which allow your objects to be accessible from other CORBA-compliant languages.

TheIDLProgrammingModel:

The IDL programming model, known as Java™ IDL, consists of both the Java CORBA ORBand the idlj compiler that maps the IDLto Java bindings that use the Java CORBA ORB, as well as aset ofAPIs, which can beexplored byselectingthe org.omgprefix from the Packagesection of the API index.

Java IDL adds CORBA (Common Object Request Broker Architecture) capability to the Java platform, providing standards-based interoperability and connectivity. Runtime components include a Java ORB for distributed computing using IIOP communication.

To use the IDL programming model, define remote interfaces using OMG Interface Definition Language (IDL), then compile the interfaces using idlj compiler. When you run the idlj compiler over your interface definition file, it generates the Java version of the interface, as well as the class code files for the stubs and skeletons that enable applications to hook into the ORB.

Portable Object Adapter (POA) : An *object adapter* is the mechanism that connects a request using an object reference with the proper code to service that request. The Portable Object Adapter, or POA, is a particular type of object adapter that is defined by the CORBA specification. The POA is designed to meet the following goals:

- Allowprogrammerstoconstructobjectimplementationsthatareportablebetweendifferent ORB products.
- Providesupport forobjects withpersistent identities.

Designing thesolution:

Here the design of how to create a complete CORBA (Common Object Request Broker Architecture) application using IDL (Interface Definition Language) to define interfaces andJava IDL compiler to generate stubs and skeletons. You can also create CORBA application by defining the interfaces in the Java programming language.

The server-side implementation generated by the idlj compiler is the *Portable Servant Inheritance Model*, also known as the POA(Portable Object Adapter) model. This document presents a sample application created using the default behavior of the idlj compiler, which usesa POA server-side model.

1. CreatingCORBAObjectsusingJavaIDL:

InordertodistributeaJavaobjectoverthenetworkusingCORBA,onehastodefineit's own CORBA-enabled interface and it implementation. This involves doing the following:

- WritinganinterfaceintheCORBA InterfaceDefinition Language
- GeneratingaJavabaseinterface,plusaJavastubandskeletonclass,usinganIDL-to-Java compiler
- Writingaserver-side implementationof theJavainterfacein Java

Interfacesin IDLaredeclaredmuchlikeinterfacesinJava.

Modules

Modules are declared in IDL using the module keyword, followed bya name for the module and

an opening brace that starts the module scope. Everything defined within the scope of this module (interfaces, constants, other modules) falls within the module and is referenced in other IDL modules using the syntax *modulename*::x. e.g.

```
//    IDL
modulejen{

   modulecorba{

      interfaceNeatExample...

   };

};
```

Interfaces

The declaration of an interface includes an interface header and an interface body. The header specifies the name of the interface and the interfaces it inherits from (if any). Here is an IDL interface header:

```
interfacePrintServer:Server{...
```

This header starts the declaration of an interface called PrintServer that inherits all the methods and data members from the Server interface.

Datamembersand methods

The interface body declares all the data members (or attributes) and methods of an interface.Data members are declared using the attribute keyword. At a minimum, the declaration includesa name and a type.

```
readonlyattribute stringmyString;
```

Themethodcanbedeclaredbyspecifyingitsname,returntype,andparameters,ataminimum. string parseString(in string buffer);

Thisdeclaresa methodcalledparseString() thatacceptsa singlestringargument andreturnsa string value.

AcompleteIDLexample

Nowlet'stieallthesebasicelementstogether.Here'sacomplete IDLexamplethatdeclares a

module within another module, which itself contains several interfaces:

```
module OS {
   moduleservices{
      interfaceServer{
         readonlyattributestringserverName; boolean
         init(in string sName);
      };

      interfacePrintable{
         booleanprint(instringheader);
      };

      interfacePrintServer:Server{

         booleanprintThis(inPrintablep);
      };
   };
};
```

Thefirstinterface,Server,has asingleread-onlystringattributeand aninit()methodthat accepts a string and returns a boolean. The Printable interface has a single print()method that accepts a string header. Finally, the PrintServer interface extends the Server interface and adds a printThis() method that accepts a Printable object and returns a boolean. In all cases, we've declared the method arguments as input-only (i.e., pass-by-value), using the in keyword.

2. TurningIDL Into Java

Once the remote interfaces in IDL are described, you need to generate Java classes that act asastartingpointforimplementingthoseremoteinterfacesinJavausinganIDL-to-Java compiler. Every standard IDL-to-Java compiler generates the following 3 Java classes from an IDL interface:

- A Java interface with the same name as the IDLinterface. This can act as the basis for aJava implementation of the interface (but you have to write it, since IDL doesn't provide any details about method implementations).

- Aholderclass whosenameis thename oftheIDLinterfacewith"Holder" appended to it

  (e.g., ServerHolder). This class is used when objects with this interface are used as out or inout arguments in remoteCORBA methods. Instead of beingpasseddirectlyinto the remote method, the object is wrapped with its holder before being passed. When a remote method has parameters that are declared as out or inout, the method has to be able to update the argument it is passed and return the updated value. The only way to guarantee this, even for primitive Java data types, is to force out and inout arguments to be wrapped in Java holder classes, which are filled with the output value of the argument when the method returns.

Theidltoj toolgenerate2 other classes:

- **A client stub class,** called _interface-nameStub, that acts as a client-side implementation of the interface and knows how to convert method requests into ORB requests that are forwardedtotheactualremoteobject.Thestubclassforaninterfacenamed Serveriscalled _ServerStub.

- **A server skeleton class,** called _interface-nameImplBase, that is a base class for a server-side implementation of the interface. The base class can accept requests for the object from the ORB and channel return values back through the ORB to the remote client. The skeleton class for an interface named Server is called _ServerImplBase.

So, in addition to generating aJavamappingofthe IDLinterfaceand some helper classes for the Java interface, the *idltoj* compiler also creates subclasses that act as an interface between a CORBA client and the ORB and between the server-side implementation and the ORB.

This creates the five Java classes: a Java version of the interface, a helper class, a holder class, a client stub, and a server skeleton.

Conclusion:

CORBA provides the network transparency, Java provides the implementation transparency. CORBAcomplements theJava™ platformbyprovidinga distributed object framework,services to support that framework, and interoperability with other languages. The Java platform complementsCORBAbyprovidingaportable,highlyproductiveimplementationenvironment.

ThecombinationofJavaandCORBAallowsyoutobuildmorescalableandmorecapable applications than can be built using the JDK alone.

Outcome:

1. Studentsunderstandarchitectureandbasicssteps forimplementationof CORBA
2. Students develop distributed application using CORBA to demonstrate object brokering for string and arithmetic operations.

FAQ:

1. What is CORBA?
2. HowCORBAworks?
3. Doesitsynchronous/Asynchronousapplication?
4. What is ORB?
5. WhatisIDLinterface?
6. WhatisObjectRequestBrokerDaemon(ORBD).
7. What is middleware.
8. Listtheexamplesofmiddleware.
9. Listtheuseof middleware.
10. ListtheapplicationsofCORBA.

ASSIGNMENTNO.4

Title:Message Passing Interface (MPI)

Aim: Developadistributedsystem,tofindsumofNelementsinanarraybydistributingN/n elements to n number of processors MPI or OpenMP. Demonstrate by displaying the intermediate sums calculated at different processors

Objective:

1. TointroduceyoutothefundamentalsofMPI.
2. Tounderstand widelyused standard for writingmessagepassingprograms.

Tools / Environment:

JavaProgrammingEnvironment,JDK1.8orhigher,MPILibrary(mpi.jar),MPJExpress (mpj.jar)

RelatedTheory:

Message passing is a popularly renowned mechanism to implement parallelism in applications;it is also called MPI. The MPI interface for Java has a technique for identifying the user and helping in lower startup overhead. It also helps in collective communication and could be executedonboth**shared memory and distributed systems**. MPJ isafamiliarJavaAPIforMPI implementation. mpiJava is the near flexible Java binding for MPJ standards.

Currently developers can produce more efficient and effective parallel applications using message passing.

Abasic prerequisitefor messagepassingisa goodcommunicationAPI. Javacomes withvarious ready-made packages for communication, notably an interface to BSD sockets, and the Remote MethodInvocation(RMI)mechanism.Theparallelcomputingworldismainlyconcernedwith `symmetric' communication, occurring in groups of interacting peers. This symmetric model of communication is captured in the successful Message Passing Interface standard (MPI).

Message-PassingInterfaceBasics:

EveryMPIprogrammustcontainthepreprocessordirective:

```
#include <mpi.h>
```

The mpi.h file contains the definitions and declarations necessary for compiling an MPIprogram.

MPI_Init initializes the execution environment for MPI. It is a “share nothing” modality in which the outcome of any one of the concurrent processes can in no way be influenced by the intermediate results of any of the other processes. Command has to be called before any other MPI call is made, and it is an error to call it more than a single time within the program. **MPI_Finalize** cleans up all the extraneous mess that was first put into place by MPI_Init.

Theprincipal weakness ofthis limitedform ofprocessingis that theprocesses on different nodes run entirely independent of each other. It cannot enable capability or coordinated computing. **To get the different processes to interact, the concept of communicators is needed.** MPI programs aremadeup of concurrent processes executingat the same time that in almost all cases

are also communicating with each other. To do this, an object called the “communicator” is provided by MPI. Thus the user may specify any number of communicators within an MPI program, each with its own set of processes. “**MPI_COMM_WORLD**” communicator contains all the concurrent processes making up an MPI program.

The size of a communicator is the number of processes that makes up the particular communicator. The following function call provides the value of the number of processes of the specified communicator:

intMPI_Comm_size(MPI_Commcomm,int _size).

The function "MPI_Comm_size” required to return the number of processes; int size. MPI_Comm_size(MPI_COMM_WORLD,&size); This will put the total number of processes in the MPI_COMM_WORLD communicator in the variable size of the process data context. Every process within the communicator has a unique ID referred to as its “rank”. MPI system automatically and arbitrarily assigns a unique positive integer value, starting with 0, to all the processes within the communicator. The MPI command to determine the process rank is:

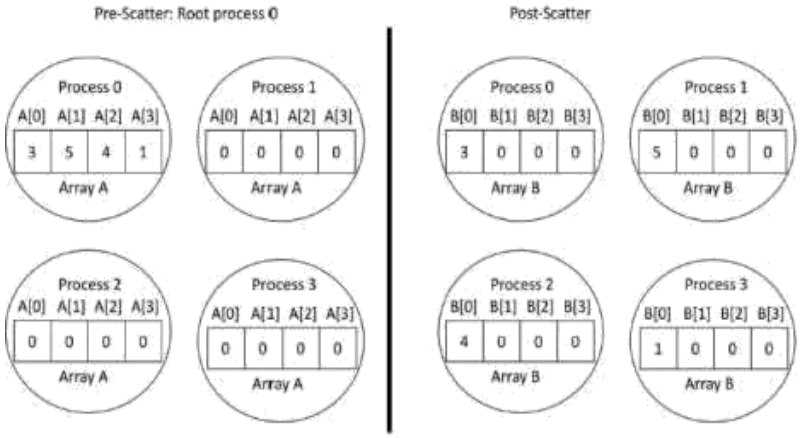
intMPI_Comm_rank(MPI_Commcomm,int_rank).

Thesendfunctionisusedbythesourceprocesstodefinethedataandestablishtheconnection of the message. The send construct has the following syntax:

intMPI_Send(void_message,intcount,MPI_Datatypedatatype,intdest,inttag, MPI_Comm comm)

The first three operands establish the data to be transferred between the source and destination processes. The first argument points to the message content itself, which may be a simple scalar oragroup ofdata.The messagedata content is described bythenext two arguments. Thesecond

operand specifies the number of data elements of which the message is composed. The third operand indicates the data type of the elements that make up the message.



The receive command (MPI_Recv) describes both the data to be transferred and the connectionto be established. The MPI_Recv construct is structured as follows:

intMPI_Recv(void_message,intcount,MPI_Datatypedatatype,intsource,inttag, MPI_Comm comm, MPI_Status _status)

Thesource field designatestherank oftheprocesssendingthe message.

Communication Collectives: Communication collective operations can dramatically expand interprocess communication from point-to-point to n-way or all-way data exchanges.

The scatter operation: Thescattercollectivecommunicationpattern,likebroadcast,sharesdata of one process (the root) with all the other processes of a communicator. But in this case it partitions a set of data of the root process into subsets and sends one subset to each of the processes. Each receiving process gets a different subset, and there are as many subsets as there are processes. In this example the send arrayis A and the receive arrayis B. B is initialized to 0. The root process (process 0 here) partitions the data into subsets of length 1 and sends each subset to a separate process.

MPJ Express is an open source Java message passing library that allows application developers to write and execute parallel applications **for multicore processors and compute clusters / clouds.** The software is distributed under the MIT (a variant of the LGPL) license. MPJ Express is a message passing library that can be used by application developers to execute their parallel Java applications on compute clusters or network of computers.

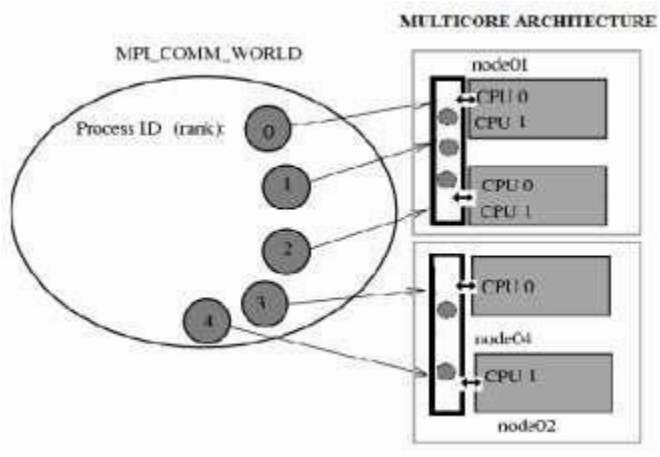
MPJExpressisessentiallyamiddlewarethatsupportscommunicationbetweenindividualprocessors of clusters. **Theprogramming model followed by MPJ Express is Single Program**

MultipleData(SPMD).

The multicore configuration is meant for users who plan to write and execute parallel Java applicationsusingMPJExpressontheirdesktops orlaptopswhichcontainssharedmemoryand multicore processors. In this configuration, users can write their message passing parallel application using MPJ Express and it will be ported automatically on multicore processors. We except that users can first develop applications on their laptops and desktops using multicore configuration, and then take the same code to distributed memory platforms

Designing thesolution:

While designing the solution, we have considered the multi-core architecture as per shown in the diagram below. The communicator has processes as per input by the user. MPI program will execute the sequence as per the supplied processes and the number of processor cores available for the execution.



Implementingthe solution:

1. ForimplementingtheMPIprograminmulti-coreenvironment,weneedtoinstallMPJ express library.
    a. DownloadMPJExpress(mpj.jar)andunpackit.
    b. SetMPJ_HOMEandPATHenvironment variables:
    c. exportMPJ_HOME=/path/to/mpj/
    d. exportPATH=$MPJ_HOME/bin:$PATH
2. WriteHelloWorldparallelJavaprogramandsave itasHelloWorld.java (Asign2.java).
3. CompileasimpleHelloWorld(Asign)parallelJavaprogram
4. RunningMPJExpressintheMulti-coreConfiguration.

Conclusion:

Thus Student develop a distributed system application, to find sum of N elements in an array by distributing N/n elements to n number of processors MPI or OpenMP. Demonstrate bydisplaying the intermediate sums calculated at different processors.

Outcome:

1. StudentslearnfundamentalsofparallelprogrammingandMPI.
2. Studentsdesignparallelalgorithms andwriteparallelprograms usingthe MPIlibrary.

FAQ:

1. Whatis use ofMPI?
2. Applicationin whichweareusingMPI?
3. Whyweareprovidingrankto processinMPI.
4. Explain MPIoperations.
5. ExplainDifferentdatatypesof MPI.
6. DrawMPIarchitecture.
7. What is MPI_ABORT?
8. What isMPI_FINALIZE
9. WhatisdifferenceMPI_ABORTandMPI_FINALIZE

ASSIGNMENTNO.5

Title:Clock Synchronization

Aim: To implement Berkeley algorithm for clock synchronization.

Objective:

1. To understand the basics of physical and Logical clock in DS.
2. To develop a node distributed system that implements Berkeley's time Synchronization algorithm.

Related Theory:

Berkeley‟s Algorithm is a clock synchronization technique used in distributed systems. The algorithmassumesthateachmachinenodeinthenetworkeitherdoesn‟thaveanaccuratetime source or doesn‟t possess an UTC server.

Algorithm:

• Anindividualnodeischosenasthemasternodefromapoolnodesinthenetwork. Thisnodeis the main node in the network which acts as a master and rest of the nodes act as slaves. Master node is chosen using a election process/leader election algorithm.

• Masternodeperiodicallypingsslavesnodesandfetchesclocktimeatthemusing Cristian‟s algorithm.

• Master node calculates average time difference between all the clock times received and the clock time given bymaster‟s system clock itself. This average time difference is added to the currenttimeatmaster‟ssystemclockandbroadcastedoverthenetwork.ScopeofImprovement

• Improvisioninaccuracyofcristian‟salgorithm.

• Ignoringsignificantoutliersincalculationofaveragetimedifference

• Incasemasternodefails/corrupts,asecondaryleadermustbeready/pre-chosentotakethe place of themaster node to reduce downtime caused due to master‟s unavailability.

• Insteadofsendingthesynchronizedtime,masterbroadcastsrelativeinversetimedifference, which leads to decrease in latency induced by traversal time in the network while time of calculation at slave node.

ScopeofImprovement

• Improvisioninaccuracy ofcristian‟salgorithm.

• Ignoringsignificantoutliersincalculationofaveragetimedifference

• Incasemasternodefails/corrupts,asecondaryleadermustbeready/pre-chosentotakethe place of themaster node to reduce downtime caused due to master‟s unavailability.

• Insteadofsendingthesynchronizedtime,masterbroadcastsrelativeinversetimedifference, which leads to decrease in latency induced by traversal time in the network while time of calculation at slave node.

Conclusion:

TheBerkeleyalgorithmisasimpleyeteffectivesolutionforclocksynchronizationindistributed systems. Its decentralized approach allows for resilience against failures, and it remains a relevant and widely used tool in ensuring accurate timekeeping between machines.

Outcome:

1. StudentslearnfundamentalofclocksynchronizationinDS.
2. StudentsimplementedBerkeleyalgorithmforclock synchronization.

FAQ:

1. Whatisdifferencebetweenlogicalclockandphysicalclock?
2. Whyis it necessarytosynchronizetheclocks indistributed real time system?
3. HowtheprincipleofBerkeleyalgorithmisusedtosynchronizetimeindistributed system?
4. Whatareother algorithmsforclock synchronizationin DS?

ASSIGNMENTNO.6

Title:Mutual Exclusion

Aim: Implementtokenringbasedmutualexclusion algorithm.

Objective:

1. Tounderstandthe conceptof starvation,Mutual Exclusionin DS.
2. ToAchieveMutualExclusion InDistributedSystembasedonTokenExchange.

RelatedTheory:

Token Ring algorithm achieves mutual exclusion in a distributed system by creating a bus network of processes. A logical ring is constructed with these processes and each process is assignedapositioninthering.Eachprocessknowswhoisnextinlineafteritself.Whenthering is initialized, process 0 is given a token. The token circulates around the ring. When a process acquires the token from its neighbor, it checks to see if it is attempting to enter a critical region. If so, the process enters the region, does all the work it needs to, and leaves the region. After it hasexited,itpassesthetokentothenextprocessinthering. Itisnotallowedtoenterthecritical region again using the same token. If a process is handed the token by its neighbor and is not interested in entering a critical region, it just passes the token along to the next process.

Advantages:

- Thecorrectnessofthisalgorithmis evident.
- Onlyoneprocesshasthetokenatanyinstant,soonlyoneprocess canbein aCSoSince the token circulates among processes in a well-defined order, starvation cannot occur.

Disadvantages

- Onceaprocessdecidesit wantstoenteraCS,atworstitwill havetowaitforeveryother process to enter and leave one critical region.
- If the token is ever lost, it must be regenerated. In fact, detectingthat it is lost is difficult, since the amount of time between successive appearances of the token on the network is notaconstant.Thefactthatthetokenhasnotbeenspottedforanhourdoesnotmeanthat it has been lost; some process may still be using it.
- Thealgorithmalsorunsintotroubleifaprocesscrashes,butrecoveryiseasierthaninthe other cases. If we require a process receiving the token to acknowledge receipt, a dead processwillbedetectedwhenitsneighbortriestogiveitthetokenandfails.Atthatpoint the dead process can beremoved from the group, and the token holder can pass the token to the next member down the line

Conclusion:

The token ring-based mutual exclusion algorithm is a well-known solution for coordinating access to shared resources in distributed systems. Its simple and efficient design ensures that onlyoneprocesscanaccessasharedresourceatatime,thuspreventingconflictsandensuring consistency. While the algorithm can suffer from potential delays and network congestion, it remains a widely used and effective solution for achieving mutual exclusion in distributed systems.

Outcome:

1. Studentslearnconceptof MutualexclusiontopreventRace conditions.
2. Studentshaveimplementedtoken ringbased mutualexclusion algorithm.

FAQ:

1. Whatisracecondition?
2. Whatisdeadlockandstarvation?
3. Whatis MutualExclusion?
4. Howto avoid mutual exclusion using

ASSIGNMENTNO.7

Title:ElectionAlgorithms

Aim: ToImplement BullyandRingalgorithmfor leader election.

Objective:

1. Toenabledistributedsystemstoselectaleaderinadecentralizedmanner,withoutrequiring a centralized control mechanism.
2. To ensure that the leader selection process is reliable and efficient, even in the presenceof failures, network delays, and other forms of uncertainty.
3. To provide a fair and deterministic method for selecting the leader, such that all nodes in the system have an equal chance of being chosen.

Tools / Environment:

JavaProgrammingEnvironment,JDK1.8,EclipseNeon(EE).

RelatedTheory:Election Algorithm:

1. Manydistributed algorithms requireaprocess toact as a coordinator.
2. Thecoordinatorcanbeanyprocess thatorganizes actionsof other processes.
3. *A*coordinator mayfail.
4. Howisanewcoordinatorchosenorelected?

Assumptions:

Eachprocesshasauniquenumbertodistinguishthem.Processesknoweachother'sprocess number.\

Therearetwotypes ofDistributed Algorithms:

1. Bully Algorithm
2. Ring Algorithm

Bully Algorithm:

A. Whenaprocess,P,noticesthatthecoordinatorisnolongerrespondingtorequests,it initiates an election.

1. PsendsanELECTION messagetoallprocesses withhighernumbers.
2. Ifnooneresponds,P wins theelectionand becomesa coordinator.
3. Ifoneofthehigher-upsanswers,ittakesover.P"sjobisdone.

B. WhenaprocessgetsanELECTIONmessagefromoneofitslower- numbered colleagues:

1. ReceiversendsanOKmessagebacktothesendertoindicatethatheisaliveandwilltake over.

2. Eventually,allprocessesgiveup apartofone,andthatoneisthenewcoordinator.

3. Thenewcoordinatorannounces*its*victorybysendingallprocessesa**CO-ORDINATOR** message telling them that it is the new coordinator.

C. Ifaprocessthat*was*previouslydowncomesback:

1. Itholdsan election.
2. Ifithappenstobethehighestprocesscurrentlyrunning,itwillwintheelectionandtake over the coordinators job.

"Biggestguy"alwayswinsandhencethenamebullyalgorithm.

Ring Algorithm

Initiation:

1. Whenaprocessnotices thatcoordinatorisnot functioning:

2. Anotherprocess(initiator)initiatestheelectionbysending"ELECTION"message(containing its own process number)

LeaderElection:

3. Initiatorsendsthemessagetoit'ssuccessor(ifsuccessorisdown,senderskipsoveritandgoes to the next member along the ring, or the one after that, until a running process is located).

4. Ateachstep, sender addsitsownprocess number to thelist inthe message.

5. Whenthemessagegetsbacktotheprocessthatstarteditall:Messagecomesbacktoinitiator. In the queue the **process with maximum ID Number wins**.

Initiatorannounces thewinnerbysendinganother messagearound the ring.

Designingthesolution:A.ForRingAlgorithm

Initiation:

1. ConsidertheProcess4understands thatProcess7isnot responding.

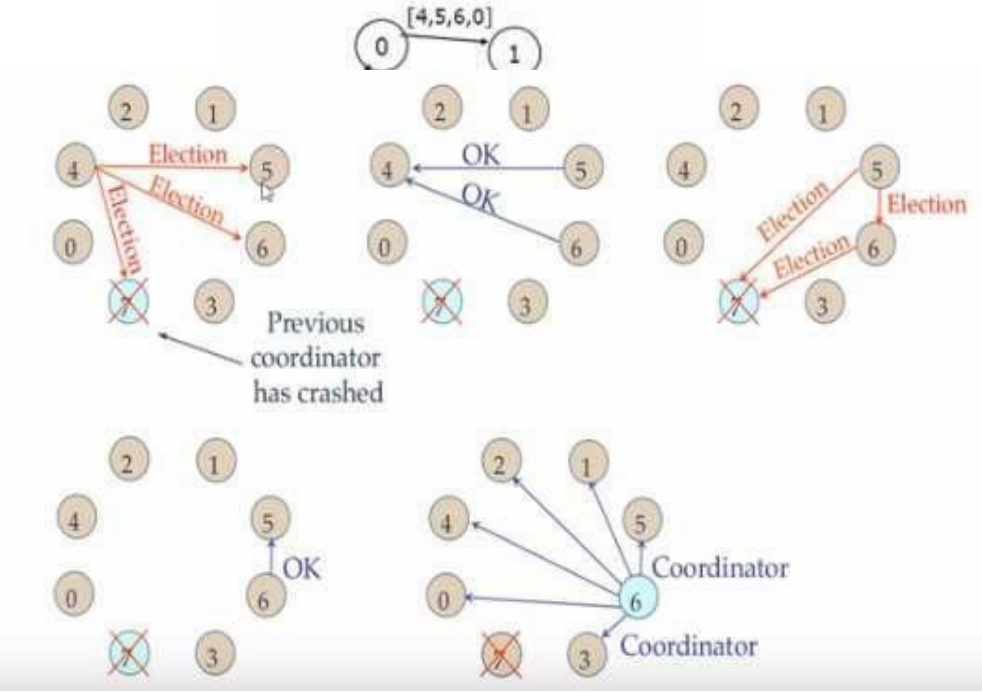
2. Process4initiatestheElectionbysending"ELECTION"messagetoit'ssuccessor(ornext

alive process) with it's ID.

LeaderElection:

3. Messagescomesback toinitiator.Heretheinitiatoris 4.

4. Initiatorannouncesthewinnerbysendinganothermessagearoundthering.Heretheprocess with highest process ID is 6. The initiator will announce that Process 6 is Coordinator.



Implementingthe solution:

ForRingAlgorithm:

1. CreatingClassfor Processwhich includes
   i) State:Active/Inactive

   ii) Index:Stores index of process.
   iii) ID:Process ID

2. ImportScannerClassforgettinginputfromConsole
3. Gettinginput fromUser fornumberofProcesses andstoretheminto objectof classes.
4. Sorttheseobjectsonthe basisofprocessid.
5. Makethe last processid as "inactive".
6. Askformenu1.Election 2.Exit

7. Ask forinitializingelectionprocess.

8. Theseinputs will be usedbyRing Algorithm.

Conclusion:

Electionalgorithms**are designed to choose a coordinator.** Wehavetwoelectionalgorithmsfor two different configurations of distributed system. **The Bully** algorithm applies to system where every process can send a message to every other process in the system and **The Ring** algorithm applies to systems organized as a ring (logically or physically). In this algorithm we assume that the link between the process are unidirectional and every process can message to the process on its right only.

Outcome:

1. StudentslearnedthefundamentalsofprocesscoordinatorelectionalgorithmsinDS
2. StudentsdevelopedBullyand Ringalgorithmfor leaderelection

FAQ:

1. Whoisprocesscoordinator?Whatareits responsibilities?
2. NeedofElectionAlgorithm?
3. Whatiscentralizedanddecentralizedalgorithm?
4. ExplainElectionworkingof algorithmforRing& Bully?
5. What is„Token"?
6. Whyalgorithm isknown as"Bully"?

ASSIGNMENTNO.8

Title:Web Services

Aim: Tocreateasimplewebserviceandwriteanydistributedapplicationtoconsumetheweb service.

Objective:

1. Tounderstandsthefundaments ofwebservices, architectureandits types.
2. Tocreateasimplewebservice.

Tools /Environment:

JavaProgrammingEnvironment,JDK8,NetbeansIDEwithGlassFishServer

RelatedTheory:

WebService:

Awebservicecanbedefinedasacollectionofopenprotocolsandstandardsfor exchanging information among systems or applications.

Aservice canbetreatedasawebserviceif:

- Theserviceisdiscoverablethroughasimplelookup It
- uses a standard XML format for messaging
- Itisavailableacrossinternet/intranet networks.
- Itis aself-describingservicethroughasimple XMLsyntax
- Theserviceis open to, and not tied to, anyoperatingsystem/programming language

Typesof Web Services:

Therearetwotypes ofweb services:

1. **SOAP**:SOAPstandsforSimpleObjectAccessProtocol.SOAPisanXMLbased industry standardprotocolfordesigninganddevelopingwebservices.Sinceit"sXML

based,it"splatformandlanguageindependent.So,ourservercanbebasedonJAVAand client can be on NET, PHP etc. and vice versa.

2. **REST**: REST (Representational State Transfer) is an architectural style for developing web services. It"s getting popularity recently because it has small learning curve when compared to SOAP. Resources are core concepts of Restful web services and they are uniquely identified by their URIs.

Webservicearchitectures:

Aspartofa webservicearchitecture,thereexistthreemajor roles.

Service Provider is the program that implements the service agreed for the web service and exposes the service over the internet/intranet for other applications to interact with.

Service Requestor is the program that interacts with the web service exposed by the Service Provider. It makes an invocation to the web service over the network to the Service Provider and exchanges information.

ServiceRegistryacts asthedirectorytostore referencestothe web services.

Thefollowingarethesteps involvedin abasic SOAP webserviceoperational behavior:

1. The client program that wants to interact with another application prepares its request content as a SOAP message.
2. Then, the client program sends this SOAP message to the server web service as an HTTP POST request with the content passed as the body of the request.
3. The web service plays a crucial role in this step by understanding the SOAP request and converting it into a set of instructions that the server program can understand.
4. Theserver programprocessesthe requestcontent asprogrammed andpreparestheoutput as the response to the SOAP request.
5. Then, the web service takes this response content as a SOAP message and reverts to the SOAP HTTP request invoked by the client program with this response.
6. The client program web service reads the SOAP response message to receive theoutcome of the server program for the request content it sent as a request.

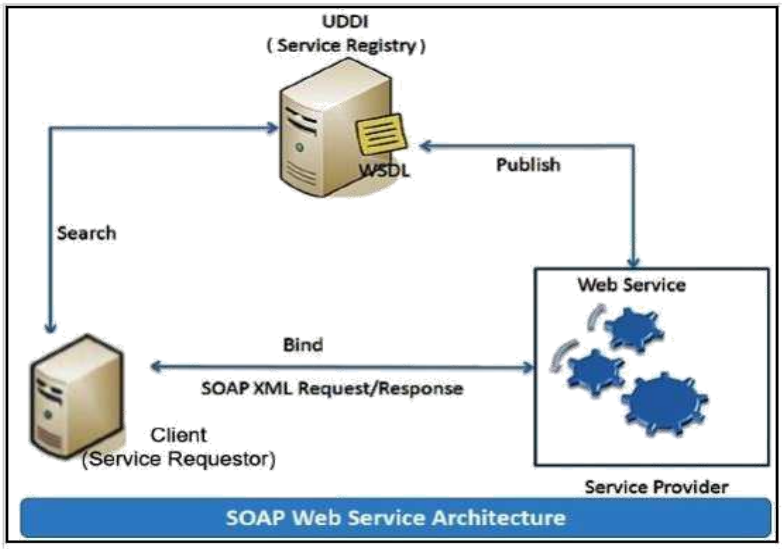
SOAPweb services

Simple Object Access Protocol (**SOAP**) is an XML-based protocol for accessing web services. It is a W3C recommendation for communication between two applications, and it is a platform- and language-independent technology in integrated distributed applications.

While XML and HTTP together make the basic platform for web services, the following are the key components of standard SOAP web services:

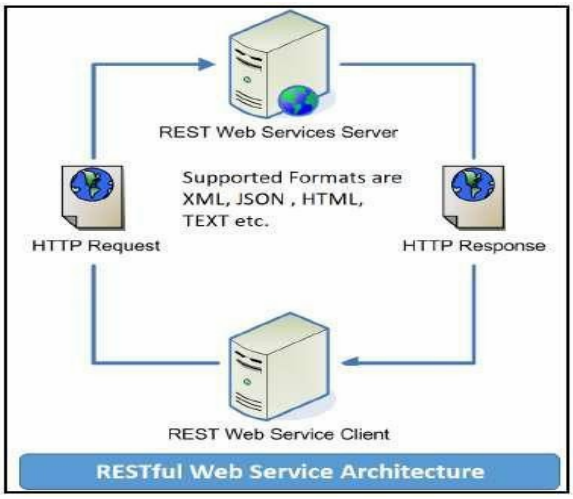
Universal Description, Discovery, and Integration (**UDDI**)*:* UDDI is an XMLbased frameworkfor describing, discovering, and integratingweb services. It acts as adirectoryof web service interfaces described in the WSDL language.

Web Services Description Language (**WSDL**)*:* WSDL is an XML document containing information about web services, such as the method name, method parameters, and how to invoke the service. WSDL is part of the UDDI registry. It acts as an interface between applications that want to interact based on web services. The following diagram shows the interaction between the UDDI, Service Provider, and service consumer in SOAP web services:



RESTfulweb services

REST stands for **Representational State Transfer**. RESTful web services are considered a performance-efficient alternativeto the SOAP web services. REST is an architectural style, not a protocol. Refer to the following diagram:



WhilebothSOAPandRESTfulsupportefficientwebservicedevelopment,thedifferencebetween these two technologies can be checked out in the following table :

SOAP	REST
SOAP is a protocol.	REST is an architectural style.
SOAP stands for Simple Object Access Protocol.	REST stands for REpresentational State Transfer.
SOAP can't use REST because it is a protocol.	REST can use SOAP web services because it is a concept and can use any protocol like HTTP, SOAP.
SOAP uses services interfaces to expose the business logic.	REST uses URI to expose business logic.
JAX-WS is the java API for SOAP web services.	JAX-RS is the java API for RESTful web services.
SOAP defines standards to be strictly followed.	REST does not define too much standards like SOAP.
SOAP requires more bandwidth and resource than REST.	REST requires less bandwidth and resource than SOAP.
SOAP defines its own security.	RESTful web services inherits security measures from the underlying transport.
SOAP permits XML data format only.	REST permits different data format such as Plain text, HTML, XML, JSON etc.
SOAP is less preferred than REST.	REST more preferred than SOAP.

Conclusion:

This assignment, described the Web services approach to the Service Oriented Architecture concept. Also, described the Java APIs for programming Web services and demonstrated examples of their use by providing detailed step-by-step examples of how to program Web services in Java.

Outcome:

1. Studentsunderstandtheconceptofwebservices.
2. Studentsdevelopedwebserviceandwriteanydistributedapplicationtoconsumetheweb service.

FAQ

1. What IsaWebService?
2. ExplainArchitectureofwebservicesw.r.toProvider,Requestor,Serviceregistryand Broker?
3. Whatis WSDL?
4. ListtypesofWebservices?
5. DifferentiatebetweenSOAPandREST?
6. Listtheexamplesofwebservices?
7. Listapplicationsofwebservices?

ASSIGNMENTNO.9

MINIPROJECTREPORT